

**LAPORAN PRAKTIKUM  
ALGORITMA & STRUKTUR DATA  
MODUL 6**



**Graph dan Tree**

**Oleh:**

**Gt. Qowita Celia Aurellizha**

**NIM. 2510817120019**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI  
2026**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA**  
**MODUL 6**

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Gt. Qowita Celia Aurellizha  
NIM : 2510817120019

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani  
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom,  
Ph.D  
NIP. 198606132015041001

## DAFTAR ISI

|                         |     |
|-------------------------|-----|
| LEMBAR PENGESAHAN ..... | ii  |
| DAFTAR ISI.....         | iii |
| DAFTAR GAMBAR .....     | v   |
| DAFTAR TABEL .....      | vi  |
| SOAL 1 .....            | 1   |
| A. Source Code .....    | 1   |
| B. Output Program.....  | 2   |
| C. Pembahasan.....      | 2   |
| SOAL 2 .....            | 4   |
| A. Source Code .....    | 4   |
| B. Output Program.....  | 5   |
| C. Pembahasan.....      | 5   |
| SOAL 3 .....            | 7   |
| A. Source Code .....    | 7   |
| B. Output Program.....  | 8   |
| C. Pembahasan.....      | 8   |
| SOAL 4 .....            | 10  |
| A. Source Code .....    | 10  |
| B. Output Program.....  | 11  |
| C. Pembahasan.....      | 11  |
| SOAL 5 .....            | 13  |

|                        |    |
|------------------------|----|
| A. Source Code .....   | 13 |
| B. Output Program..... | 16 |
| C. Pembahasan.....     | 16 |
| TAUTAN GIT.....        | 18 |

## **DAFTAR GAMBAR**

|                                             |    |
|---------------------------------------------|----|
| Gambar 1. Screenshot Hasil Run Soal 1 ..... | 2  |
| Gambar 2. Screenshot Hasil Run Soal 2 ..... | 5  |
| Gambar 3. Screenshot Hasil Run Soal 3 ..... | 8  |
| Gambar 4. Screenshot Hasil Run Soal 4 ..... | 11 |
| Gambar 5. Screenshot Hasil Run Soal 5 ..... | 16 |

## DAFTAR TABEL

|                                   |    |
|-----------------------------------|----|
| Tabel 1. Source Code Soal 1 ..... | 1  |
| Tabel 2. Source Code Soal 2 ..... | 4  |
| Tabel 3. Source Code Soal 3 ..... | 7  |
| Tabel 4. Source Code Soal 4 ..... | 10 |
| Tabel 5. Source Code Soal 5 ..... | 13 |

## SOAL 1

Diberikan sebuah directed weighted graph yang terdiri dari  $N$  vertex dan  $M$  edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge. Setiap edge ditulis sebagai  $U,V,W$ , yang berarti terdapat edge berarah dari vertex  $U$  menuju vertex  $V$  dengan bobot  $W$ . Karena graph bersifat berarah, edge  $U,V,W$  tidak otomatis berarti terdapat edge dari  $V$  menuju  $U$ . Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ , maka nilai matrix pada baris  $i$  dan kolom  $j$  adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

### A. Source Code

Tabel 1. Source Code Soal 1

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N;
6     cin >> N;
7
8     vector<char> labels(N);
9     map<char, int> idx;
10    for (int i = 0; i < N; i++) {
11        cin >> labels[i];
12        idx[labels[i]] = i;
13    }
14
15    vector<vector<int>> matrix(N, vector<int>(N, 0));
16
17    int M;
18    cin >> M;
19    for (int i = 0; i < M; i++) {
20        char u, v; int w;
21        cin >> u >> v >> w;
22        if (idx.count(u) && idx.count(v)) {
23            matrix[idx[u]][idx[v]] = w;
24        }
25    }
```

```

26
27     cout << "Adjacency Matrix:" << endl;
28
29     cout << " ";
30     for (int i = 0; i < N; i++)
31         cout << " " << labels[i];
32     cout << endl;
33
34     for (int i = 0; i < N; i++) {
35         cout << labels[i];
36         for (int j = 0; j < N; j++)
37             cout << " " << matrix[i][j];
38         cout << endl;
39     }
40
41     return 0;
42 }

```

## B. Output Program

```

PS C:\Users\ASUS\OneDrive\Documents\Github\module-6-graph-and-tree-inipunyaCELI> g++ soal1.cpp -o soal1; ./soal1.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\ASUS\OneDrive\Documents\Github\module-6-graph-and-tree-inipunyaCELI>

```

Gambar 1. Screenshot Hasil Run Soal 1

## C. Pembahasan

Soal 1 program digunakan untuk mengubah data graph yang diberikan dalam bentuk daftar edge menjadi adjacency matrix. Graph yang digunakan bersifat berarah (directed) dan memiliki bobot (weighted) sedangkan setiap vertex diberi label berupa huruf.

Program dimulai dengan membaca N vertex dan labelnya ke dalam `vector<char>labels`. Dipakai juga `map<char, int> idx` untuk memetakan setiap label karakter ke indeks integer yang digunakan untuk baris dan kolom pada



matrix. Penggunaan `map<char, int>` memudahkan program untuk menentukan posisi setiap vertex di matriks, karena label vertex berupa karakter sedangkan indeks matriks menggunakan angka. Matrix  $N \times N$  di inisialisasi dengan nilai 0 yang artinya tidak ada edge, setiap edge  $U \ V \ W$  yang dibaca lalu diproses dengan mencari indeks dari label  $U$  dan  $V$  menggunakan `idx`, dan mengisi `matrix[idx[U][idx[V]]] = W`. Karena graph bersifat berarah yaitu hanya satu arah yang di isi sehingga kebalikannya tetap 0.

Matriks berukuran  $N \times N$  dibuat terlebih dahulu dengan nilai awal 0 setelah itu program membaca seluruh edge yang ada dan mengisi posisi yang sesuai di matriks. Karena matriks disimpan penuh, kebutuhan memorinya mengikuti ukuran matriks tersebut makanya kompleksitas waktunya adalah  $O(N^2)$  untuk inisialisasi matrix dan  $O(M)$  untuk membaca  $M$  edge maka total  $O(N^2 + M)$ . Kompleksitas ruang adalah  $O(N^2)$  untuk menyimpan matrix.

## SOAL 2

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran  $N \times N$ . Setiap vertex memiliki label berupa satu karakter unik. Nilai matrix pada baris  $i$  dan kolom  $j$  menunjukkan bobot edge dari vertex ke- $i$  menuju vertex ke- $j$ . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ . Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

### A. Source Code

Tabel 2. Source Code Soal 2

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N;
6     cin >> N;
7
8     vector<char> labels(N);
9     for (int i = 0; i < N; i++) cin >> labels[i];
10
11     vector<vector<int>> matrix(N, vector<int>(N));
12     for (int i = 0; i < N; i++)
13         for (int j = 0; j < N; j++)
14             cin >> matrix[i][j];
15
16     cout << "Adjacency List:" << endl;
17     for (int i = 0; i < N; i++) {
18         cout << labels[i] << " -> ";
19         bool hasEdge = false;
20         bool first = true;
21         for (int j = 0; j < N; j++) {
22             if (matrix[i][j] > 0) {
23                 if (!first) cout << ", ";
24                 cout << "(" << labels[j] << ", " <<
25 matrix[i][j] << ")";
```

```

26         first = false;
27         hasEdge = true;
28     }
29 }
30 if (!hasEdge) cout << "-";
31 cout << endl;
32 }
33
34 return 0;
35 }

```

## B. Output Program

```

PS C:\Users\ASUS\OneDrive\Documents\github\module-6-graph-and-tree-inipanyacell> g++ soal2.cpp -o soal2; ./soal2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -

```

Gambar 2. Screenshot Hasil Run Soal 2

## C. Pembahasan

Soal 2 kebalikan dari soal 1 yaitu mengubah representasi adjacency matrix menjadi adjacency list. Pada soal ini, data graph yang awalnya berbentuk adjacency matrix diubah menjadi adjacency list supaya hubungan antar vertex dapat ditampilkan dengan mudah.

Program membaca  $N$  vertex ke dalam `vector<char>` labels kemudian membaca matrix  $N \times N$  secara langsung. Setiap vertex  $i$  program akan melakukan scan pada seluruh baris ke- $i$  dari matrix, jika nilai  $matrix[i][j] > 0$  maka terdapat edge dari vertex  $labels[i]$  menuju  $labels[j]$  dengan bobot  $matrix[i][j]$  dan di cetak dalam format  $(labels[j], \text{bobot})$ . Jika tidak ada satupun edge yang ditemukan maka baris  $i$  dicetak tanda  $(-)$ . Urutan pada output mengikuti urutan label pada input yang sesuai dengan urutan indeks pada matrix.

Program memeriksa seluruh isi matriks satu per satu untuk menentukan apakah terdapat edge di posisi tersebut, jika ditemukan nilai lebih dari 0 maka vertex tujuan dan bobotnya akan ditampilkan makanya kompleksitas waktu program adalah  $O(N^2)$  karena setiap elemen matrix diperiksa satu kali.

### SOAL 3

Sebuah labirin direpresentasikan sebagai grid berukuran  $R \times C$ . Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

#### A. Source Code

*Tabel 3. Source Code Soal 3*

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int R, C;
7      cin >> R >> C;
8
9      vector<vector<int>> grid(R, vector<int>(C));
10     for (int i = 0; i < R; i++)
11         for (int j = 0; j < C; j++)
12             cin >> grid[i][j];
13
14     int sr, sc, fr, fc;
15     cin >> sr >> sc >> fr >> fc;
16
17     vector<vector<int>> dist(R, vector<int>(C, -1));
18     queue<pair<int, int>> q;
19     dist[sr][sc] = 0;
20     q.push({sr, sc});
21
```

```

22     int dx[] = {-1, 1, 0, 0};
23     int dy[] = {0, 0, -1, 1};
24
25     while (!q.empty())
26     {
27         auto [x, y] = q.front();
28         q.pop();
29         for (int d = 0; d < 4; d++)
30         {
31             int nx = x + dx[d], ny = y + dy[d];
32             if (nx >= 0 && nx < R && ny >= 0 && ny <
33 C && grid[nx][ny] == 0 && dist[nx][ny] == -1)
34             {
35                 dist[nx][ny] = dist[x][y] + 1;
36                 q.push({nx, ny});
37             }
38         }
39     }
40
41     cout << dist[fr][fc] << endl;
42     return 0;
43 }

```

## B. Output Program

```

PS C:\Users\ASUS\OneDrive\Documents\GitHub\module-6-graph-and-tree-inipunyaCELI> g++ soal3.cpp -o soal3; ./soal3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 3. Screenshot Hasil Run Soal 3

## C. Pembahasan

Soal 3 meminta pencarian jarak terpendek di labirin berbentuk grid  $R \times C$  menggunakan algoritma Breadth-First Search (BFS). Sel bernilai 0 merupakan jalan yang bisa dilewati sedangkan sel bernilai 1 adalah dinding yang tidak bisa dilewati. Pergerakannya hanya diperbolehkan ke empat arah yaitu atas, bawah, kiri, dan kanan.

Algoritma BFS digunakan karena pencarian dilakukan secara bertahap dari posisi awal, semua posisi yang berjarak satu langkah akan diperiksa terlebih dahulu kemudian dilanjutkan ke posisi yang berjarak dua langkah dan seterusnya. Dengan cara ini jumlah langkah minimum menuju tujuan dapat ditemukan, maka ketika posisi tujuan pertama kali dikunjungi jarak yang tercatat pasti jarak minimum.

Pada program, queue digunakan untuk menyimpan posisi yang akan diperiksa berikutnya. Sementara itu, array *dist* digunakan untuk mencatat jumlah langkah dari posisi awal ke setiap sel yang berhasil dikunjungi. Sel start yang dimasukkan ke queue dengan  $dist[sr][sc] = 0$ . Setiap iterasi koordinat diambil dari depan queue dan keempat tetangganya diperiksa. Jika tetangganya valid dalam batas grid, bernilai 0, dan belum dikunjungi maka  $dist[nx][ny] = dist[x][y] + 1$  dan koordinat dimasukkan ke queue. Jika posisi tujuan tidak bisa dicapai maka  $dist[fr][fc]$  tetap  $-1$ .

Kompleksitas waktu Breadth-First Search (BFS) adalah  $O(R \times C)$  karena setiap sel dikunjungi paling banyak satu kali. Kompleksitas ruangnya juga  $O(R \times C)$  untuk menyimpan array *dist* dan queue.

## SOAL 4

Diberikan sebuah labirin berukuran  $R \times C$  dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif. Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

### A. Source Code

Tabel 4. Source Code Soal 4

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int R, C, sr, sc, fr, fc;
5  vector<vector<int>> grid;
6  vector<vector<bool>> visited;
7  int count_paths = 0;
8
9  int dx[] = {-1, 1, 0, 0};
10 int dy[] = {0, 0, -1, 1};
11
12 void dfs(int x, int y)
13 {
14     if (x == fr && y == fc)
15     {
16         count_paths++;
17         return;
18     }
19     for (int d = 0; d < 4; d++)
20     {
21         int nx = x + dx[d], ny = y + dy[d];
22         if (nx >= 0 && nx < R && ny >= 0 && ny < C &&
23 grid[nx][ny] == 0 && !visited[nx][ny])
24         {
25             visited[nx][ny] = true;
```



```

26         dfs(nx, ny);
27         visited[nx][ny] = false;
28     }
29 }
30 }
31
32 int main()
33 {
34     cin >> R >> C;
35     grid.assign(R, vector<int>(C));
36     for (int i = 0; i < R; i++)
37         for (int j = 0; j < C; j++)
38             cin >> grid[i][j];
39     cin >> sr >> sc >> fr >> fc;
40
41     visited.assign(R, vector<bool>(C, false));
42     visited[sr][sc] = true;
43     dfs(sr, sc);
44
45     cout << count_paths << endl;
46     return 0;
47 }

```

## B. Output Program

The screenshot shows a terminal window with the following output:

```

PS C:\Users\VASUS\OneDrive\Documents\Github\module-6-graph-and-tree-in-jupyter\cell1> g++ soal4.cpp -o soal4; ./soal4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

The output represents a 5x6 maze grid where 0 is an open cell and 1 is a wall. The start cell is at (4, 5) and the finish cell is at (4, 4). The program outputs the number of paths from start to finish, which is 4.

Gambar 4. Screenshot Hasil Run Soal 4

## C. Pembahasan

Soal 4 meminta penghitungan semua jalur sederhana dari posisi start ke finish di labirin menggunakan algoritma Depth-First Search (DFS) rekursif dengan backtracking. Jalur sederhananya jika tidak mengunjungi sel yang sama lebih dari sekali dalam satu jalur.

Pada soal ini digunakan algoritma DFS karena program harus mencoba seluruh kemungkinan jalur dari titik awal menuju titik tujuan. Setelah proses pencarian pada suatu jalur selesai, status visited dikembalikan seperti awal supaya sel tersebut masih bisa digunakan saat program mencoba jalur lainnya.

Fungsi `dfs(x, y)` dipanggil secara rekursif. Jika koordinat  $(x, y)$  sama dengan  $(fr, fc)$ , berarti satu jalur valid ditemukan dan `count_paths` di inkremen, untuk setiap arah yang valid dalam batas grid, bernilai 0, dan belum dikunjungi, `visited[nx][ny]` diset true sebelum rekursi dan dikembalikan ke false setelah rekursi selesai. Ini merupakan backtracking yang sel tersebut digunakan jalur lain.

Karena program mencoba banyak kemungkinan jalur yang berbeda jumlah proses yang dilakukan bisa menjadi sangat besar ketika ukuran grid semakin besar, ukuran grid pada soal dibatasi agar program tetap dijalankan dengan baik makanya kompleksitas waktu dalam worst case adalah  $O(4^{(R \times C)})$  karena setiap sel memiliki 4 pilihan arah tetapi dalam praktiknya jauh lebih kecil karena dibatasi dinding, batas grid, dan visited array. Soal membatasi ukuran grid 7x7 supaya tidak terjadi timeout.

## SOAL 5

Diberikan  $N$  bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree. Setelah RBTree terbentuk, diberikan  $Q$  query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

### A. Source Code

*Tabel 5. Source Code Soal 5*

|    |                                                                      |
|----|----------------------------------------------------------------------|
| 1  | <code>#include &lt;bits/stdc++.h&gt;</code>                          |
| 2  | <code>#include "RedBlackTree.h"</code>                               |
| 3  | <code>using namespace std;</code>                                    |
| 4  |                                                                      |
| 5  | <code>void preorder(const RedBlackTree::Node *node, const</code>     |
| 6  | <code>RedBlackTree::Node *nil, vector&lt;int&gt; &amp;result)</code> |
| 7  | <code>{</code>                                                       |
| 8  | <code>    if (node-&gt;isNil)</code>                                 |
| 9  | <code>        return;</code>                                         |
| 10 | <code>    result.push_back(node-&gt;key);</code>                     |
| 11 | <code>    preorder(node-&gt;left, nil, result);</code>               |
| 12 | <code>    preorder(node-&gt;right, nil, result);</code>              |
| 13 | <code>}</code>                                                       |
| 14 |                                                                      |
| 15 | <code>void inorder(const RedBlackTree::Node *node, const</code>      |
| 16 | <code>RedBlackTree::Node *nil, vector&lt;int&gt; &amp;result)</code> |
| 17 | <code>{</code>                                                       |
| 18 | <code>    if (node-&gt;isNil)</code>                                 |
| 19 | <code>        return;</code>                                         |
| 20 | <code>    inorder(node-&gt;left, nil, result);</code>                |
| 21 | <code>    result.push_back(node-&gt;key);</code>                     |
| 22 | <code>    inorder(node-&gt;right, nil, result);</code>               |

```

23 }
24
25 void postorder(const RedBlackTree::Node *node, const
26 RedBlackTree::Node *nil, vector<int> &result)
27 {
28     if (node->isNil)
29         return;
30     postorder(node->left, nil, result);
31     postorder(node->right, nil, result);
32     result.push_back(node->key);
33 }
34
35 void printTraversal(const string &label, const
36 vector<int> &v)
37 {
38     cout << label << " : ";
39     for (int i = 0; i < (int)v.size(); i++)
40     {
41         if (i)
42             cout << " ";
43         cout << v[i];
44     }
45     cout << endl;
46 }
47
48 int main()
49 {
50     int N;
51     cin >> N;
52
53     RedBlackTree rbt;
54     set<int> seen;
55     for (int i = 0; i < N; i++)
56     {
57         int x;
58         cin >> x;
59         if (!seen.count(x))
60         {
61             seen.insert(x);
62             rbt.insert(x);
63         }
64     }
65

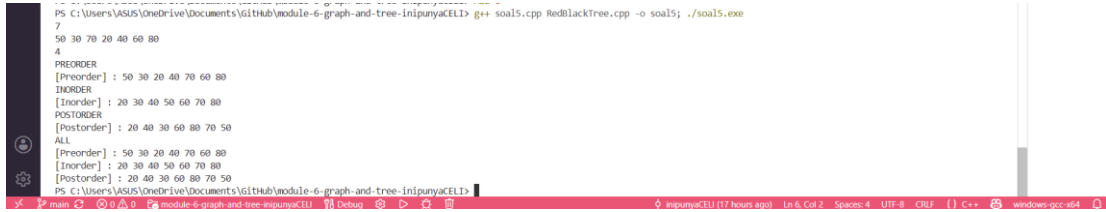
```

```

66     int Q;
67     cin >> Q;
68     while (Q-->0)
69     {
70         string query;
71         cin >> query;
72
73         if (rbt.empty())
74         {
75             cout << "Tree kosong. Tidak ada yang bisa
76 ditampilkan." << endl;
77             continue;
78         }
79
80         vector<int> pre, in, post;
81         preorder(rbt.root(), rbt.nil(), pre);
82         inorder(rbt.root(), rbt.nil(), in);
83         postorder(rbt.root(), rbt.nil(), post);
84
85         if (query == "PREORDER")
86         {
87             printTraversal("[Preorder]", pre);
88         }
89         else if (query == "INORDER")
90         {
91             printTraversal("[Inorder]", in);
92         }
93         else if (query == "POSTORDER")
94         {
95             printTraversal("[Postorder]", post);
96         }
97         else if (query == "ALL")
98         {
99             printTraversal("[Preorder]", pre);
100             printTraversal("[Inorder]", in);
101             printTraversal("[Postorder]", post);
102         }
103     }
104     return 0;
105 }

```

## B. Output Program



```
PS C:\Users\VASUS\OneDrive\Documents\github\module-6-graph-and-tree-inipunyaCELL> g++ soal5.cpp RedBlackTree.cpp -o soal5.exe
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\Users\VASUS\OneDrive\Documents\github\module-6-graph-and-tree-inipunyaCELL>
```

Gambar 5. Screenshot Hasil Run Soal 5

## C. Pembahasan

Soal 5 menggunakan struktur data Red-Black Tree (RBTree) yang sudah disediakan melalui file Red-Black Tree.h dan Red-Black Tree.cpp. Tugas program adalah memasukkan N bilangan bulat ke dalam Red-Black Tree (RBTree) lalu menjawab Q query traversal yaitu PREORDER, INORDER, POSTORDER, atau ALL.

Program menggunakan struktur data Red-Black Tree, struktur ini digunakan untuk menyimpan data secara terurut sehingga proses traversal dapat dilakukan dengan mudah. Red-Black Tree (RBTree) menggunakan sentinel node nil (berwarna hitam) sebagai pengganti nullptr, sehingga pengecekan batas menggunakan `node->isNil`.

Tiga fungsi traversal diimplementasikan secara rekursif. Preorder mengunjungi root terlebih dahulu, kemudian subtree kiri lalu subtree kanan (Root-left-Right). Inorder mengunjungi subtree kiri, kemudian root, lalu subtree kanan (Left-Root-Right) pada BST akan menghasilkan urutan nilai dari kecil ke besar. Postorder mengunjungi subtree kiri, kemudian subtree kanan, lalu root (Left-Right-Root). Proses rekursi akan berhenti ketika node yang diperiksa merupakan node nil sehingga program tidak melanjutkan penelusuran ke bagian yang tidak memiliki data.

Penanganan duplikat dilakukan menggunakan `set<int> seen` sebelum memanggil `rbt.insert()`. Jika bilangan sudah ada di `seen` maka tidak dimasukkan ke RBTtree. Jika setelah semua insert Rbtree kosong (`rbt.empty()`) maka setiap query dijawab dengan pesan "Tree kosong tidak ada yang bisa ditampilkan".

Saat traversal dilakukan setiap node pada tree akan dikunjungi satu kali sesuai urutan traversal yang dipilih. Hasil traversal kemudian disimpan ke dalam vector sebelum ditampilkan makanya kompleksitas waktu setiap traversal adalah  $O(N)$  karena setiap node dikunjungi tepat satu kali. Kompleksitas ruang  $O(N)$  untuk rekursi stack dan penyimpanan hasil traversal.

## TAUTAN GIT

[DSA26-ULM/module-6-graph-and-tree-inipunyaCELI: dsa26-ulm-classroom-3c2beb-module-6-graph-and-tree-module-6-graph-and-tree created by GitHub Classroom](#)